

19. Pipelining (4)

Reordering Code

- Example in C:

```
a = b + e;
```

```
c = b + f;
```

- In assembly, this might be:

```
lw x1, 0(x31)    // load b
```

```
lw x2, 4(x31)    // load e
```

```
add x3, x1, x2    // b + e
```

Hazard

```
sw x3, 12(x31)    // store a
```

```
lw x4, 8(x31)    // load f
```

```
add x5, x1, x4    // b + f
```

Hazard

```
sw x5, 16(x31)    // store c
```

Reordering Code

- Both add instructions have hazards. Even with forwarding, there is a stall before each (13 cycles)

	1	2	3	4	5	6	7	8	9	10	11
lw	IF	ID	EX	MEM	WB						
lw		IF	ID	EX	MEM	WB					
nop			⌘	⌘	⌘	⌘	⌘				
add				IF	ID	EX	MEM	WB			
sw					IF	ID	EX	MEM	WB		
lw						IF	ID	EX	MEM	WB	
nop							⌘	⌘	⌘	⌘	⌘
add								IF	ID	EX	MEM
sw									IF	ID	EX

Reordering Code

- The compiler could reorder the code:

```
lw x1, 0(x31)    // load b
lw x2, 4(x31)    // load e
lw x4, 8(x31)    // load f
add x3, x1, x2   // b + e
add x5, x1, x4   // b + f
sw x3, 12(x31)   // store a
sw x5, 16(x31)   // store c
```

Move this load forward

- This eliminates the stalls

Reordering Code

- This now takes 11 cycles

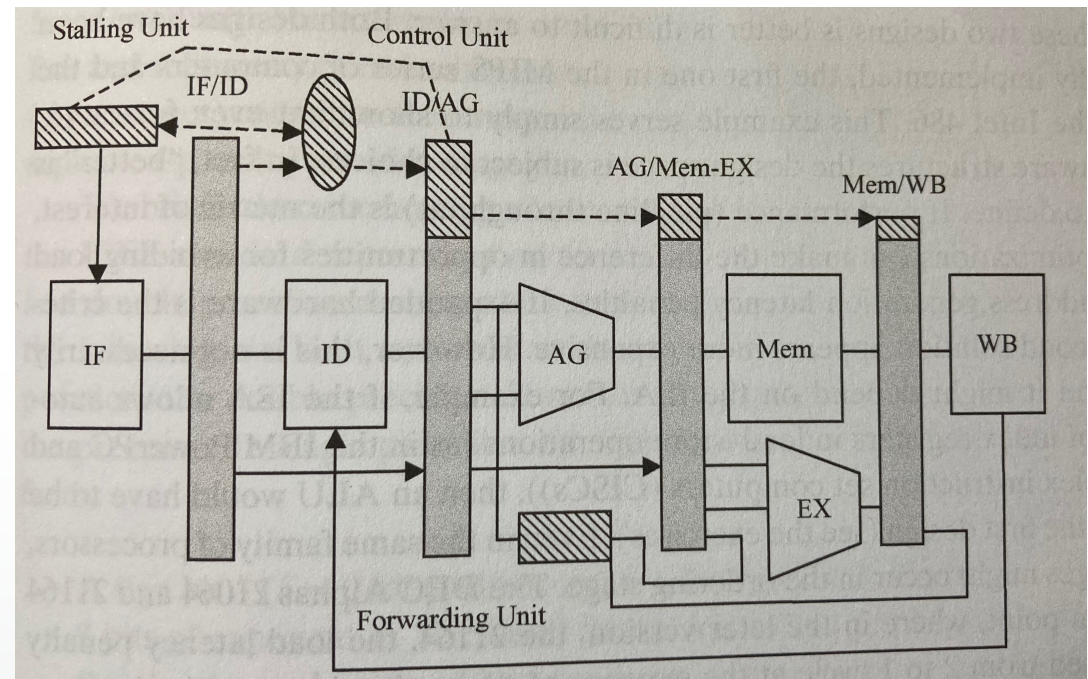
	1	2	3	4	5	6	7	8	9	10
lw	IF	ID	EX	MEM	WB					
lw		IF	ID	EX	MEM	WB				
lw			IF	ID	EX	MEM	WB			
add				IF	ID	EX	MEM	WB		
add					IF	ID	EX	MEM	WB	
sw						IF	ID	EX	MEM	WB
sw							IF	ID	EX	MEM

- 2nd add is also moved forward to allow the forwarding to work
- It's not going to be possible to do this every time – some stalls are inevitable

Another Approach

- Redesign the pipeline
 - EX is moved to the 4th stage
 - This avoids the load-execute stall
 - But introduces a new dependency

```
add x5, x6, x7  
lw x9, 0(x5)
```



From: Microprocessor Architecture, Baer

Interrupts and Exceptions

- One further control hazard!
 - Unexpected events
 - Interrupts generally come from outside (I/O)
 - Exceptions are generally internal (undefined instructions, overflow, etc.)
 - Change of privilege
 - May not distinguish at CPU level
 - Cache misses (see later lecture) are similar to exceptions, but handled separately
- Assumptions
 - Take action (almost) immediately
 - Operating system will handle the event
- Will look at interrupts in more detail later (if time allows)

Exceptions in RISC-V/MIPS

- Most computers support *precise interrupts*
 - If there are 5 instructions in the pipeline, which is the one that is interrupted?
 - We assume the instruction currently in the EX stage
- Save Program Counter for that instruction
- Complete instructions ahead (in MEM and WB stages)
- Flush pipeline for current instruction and those behind it (IF, ID, EX stages)
- Jump to interrupt handler routine

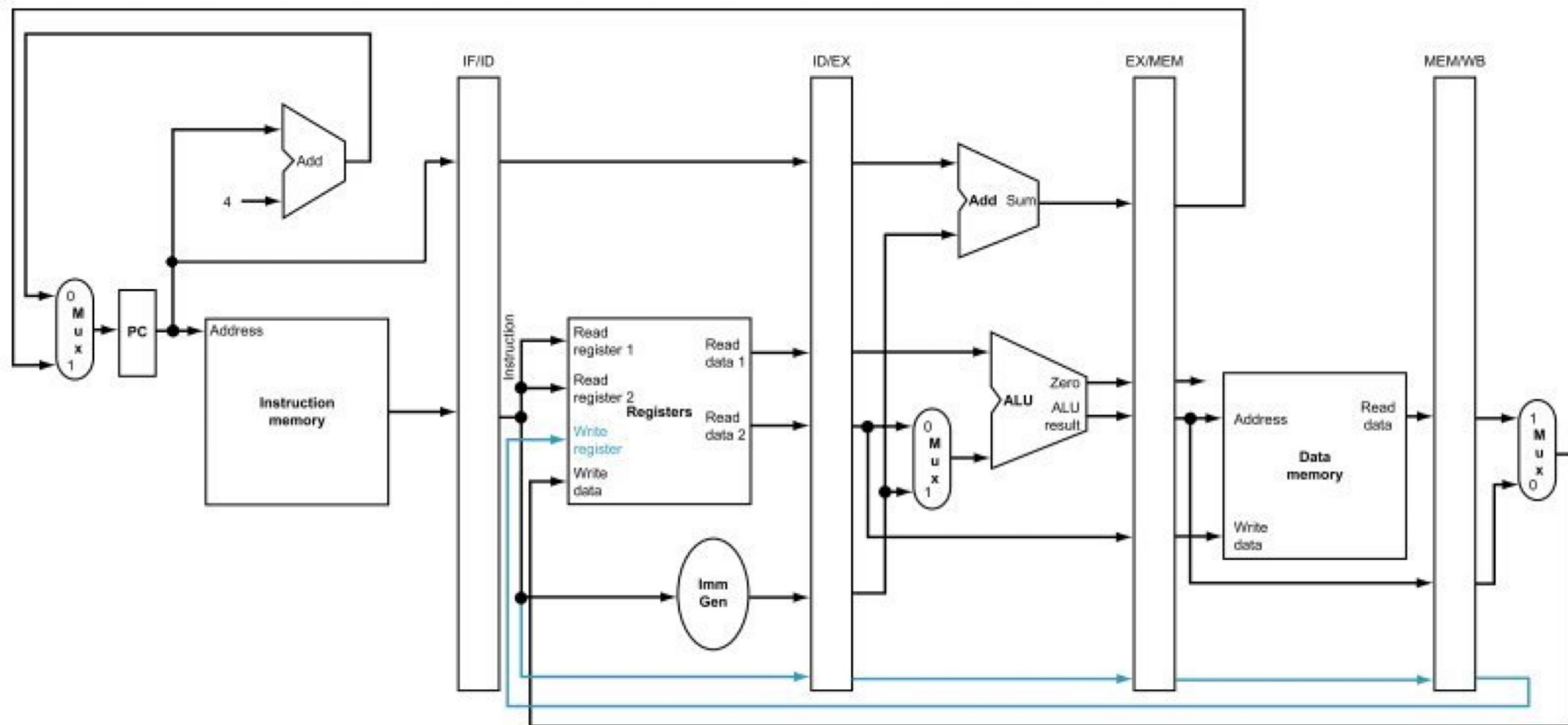
Exceptions

- RISC-V adds 2 registers
 - SEPC *Supervisor Exception Program Counter* (EPC in MIPS) – 32 bits
 - Holds the address of the affected instruction
 - SCAUSE *Supervisor Exception Cause Register*
 - Records cause of the exception
- RV32I instruction set has 4 relevant instructions (all I-type)
 - ECALL – Environment call exception to invoke OS
 - EBREAK – Exception to invoke the debugger
 - SRET – Supervisor Exception Return
 - WRI – Wait for Interrupt (do nothing)

Pipeline Registers

- 4 pipeline registers (in 5 stage pipeline)
 - IF/ID, ID/EX, EX/MEM, MEM/WB
 - Wide enough to hold all the bits needed for subsequent stages
 - One clock drives PC + all registers
 - If register file is synchronous, the read data outputs would be part of the ID/EX register
 - We don't want 2 synchronous registers there
 - For example:
 - IF/ID would be 64 bits (Instruction + PC)
 - As we will see, these registers need more bits for control, etc.

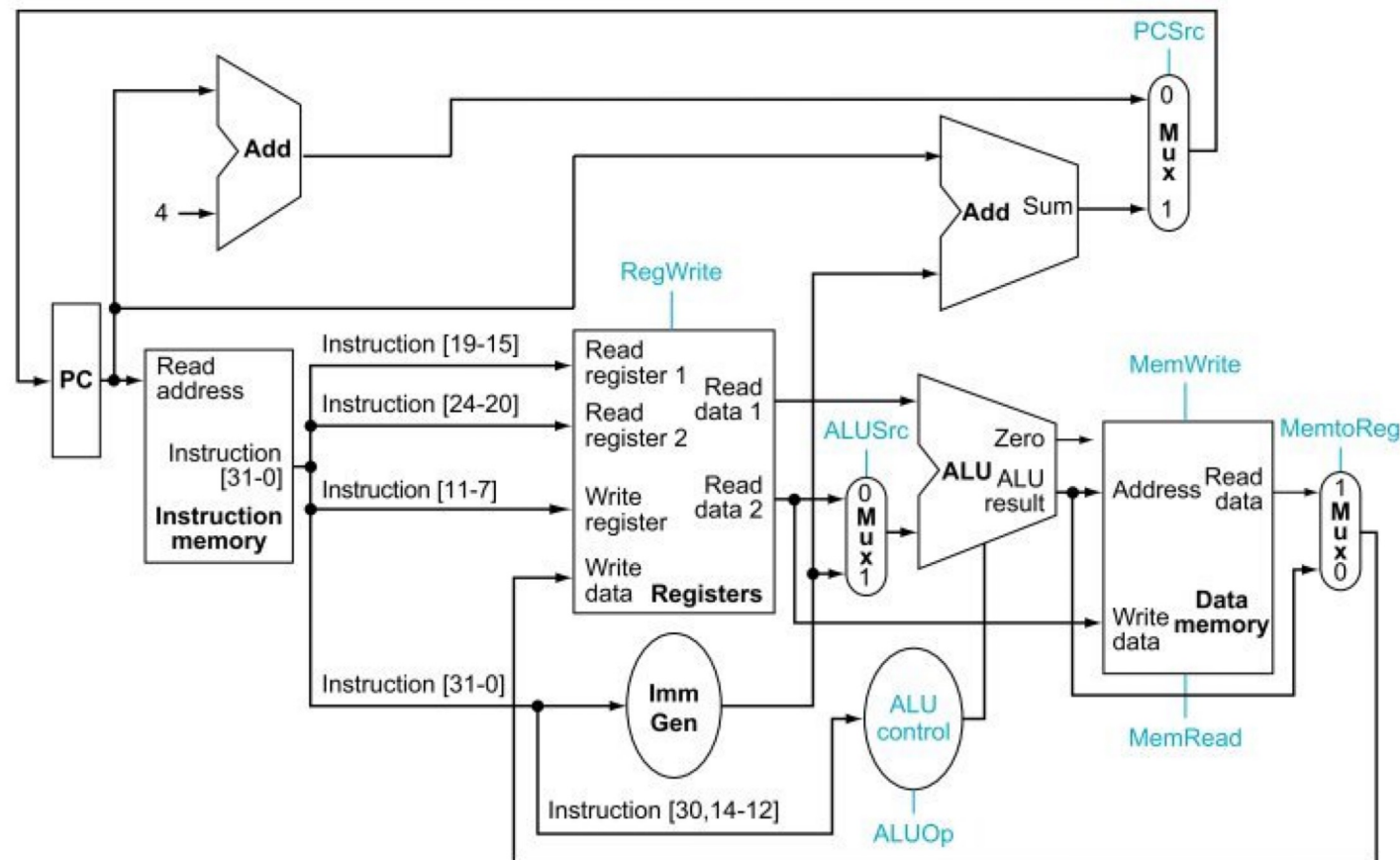
Pipeline with load register address



5 bits – rd address

Control Signals

- Hardware elements need control signals



Control Unit

- Combinational Logic to decode opcode, funct7, funct3

